# TOP-UP & TOURNAMENT SYSTEM

Product Requirements, User Flows, Admin Controls, API Outline & Security Checklist

Version 1.0 | Prepared: 11 October 2026

**Document purpose:** A practical build specification for product owners, UI designers, backend developers, QA, and operations.

**Important implementation note:** This is a technology-neutral specification, not a claim that the features have already been implemented. Confirm project stack, provider APIs, and legal requirements before production launch.

## 1. Purpose and Scope

- This document defines a professional Top-Up and Tournament system for a gaming platform. It is a build specification for the product UI, backend, database, admin dashboard, payment verification, tournament operations, and security.
- Assumptions: the platform may support both wallet-based top-ups and direct game-currency top-ups; free and paid tournaments; web and Android clients. Confirm game titles, payment providers, and technology stack before implementation.

## 2. Main User Areas

- Home: featured top-up offers, upcoming tournaments, live tournaments, wallet balance, recent orders, and announcements.
- Top-Up: select a game or wallet package, enter player details when required, review price and fees, choose payment method, confirm the order, and track status.
- Wallet: available balance, pending balance if applicable, ledger/transaction history, deposit records, refunds, and prize credits.
- Tournaments: browse and filter tournaments; view rules, schedule, format, entry fee, prize pool, slots, and registration status; join or cancel when permitted.
- My Matches: show registered tournaments, team members, match schedule, room details when released, standings, and results.
- Profile and Support: verified contact details, account security, help tickets, dispute status, and responsible-play / eligibility notices where relevant.

## 3. Top-Up User Flow

- 1. User chooses Wallet Top-Up or Game Top-Up.
- 2. For a game top-up, user selects the game and package, then enters the required Player ID / server ID. Validate field formats where possible.
- 3. Display item quantity, base price, any clearly disclosed fees, final amount, delivery estimate, and cancellation/refund terms.
- 4. Create a unique order in PENDING_PAYMENT state before redirecting to a payment provider or displaying approved payment instructions.
- 5. Confirm payment only from a verified server-to-server webhook or a trusted provider status API. Never trust a client-side success screen or screenshot alone.
- 6. After verified payment, atomically mark the payment successful and either credit the wallet ledger or create a game delivery job.
- 7. Delivery workers must be idempotent. Retry temporary failures without creating duplicate credits or duplicate deliveries.
- 8. Show status updates: Pending Payment, Paid, Processing, Delivered, Failed, Refund Pending, Refunded, or Under Review.

## 4. Top-Up Rules and Edge Cases

- Use a unique internal order ID and provider transaction ID; enforce uniqueness on provider transaction IDs.
- Do not credit balances twice if a webhook is repeated. Verify webhook signatures, timestamps, and event IDs.
- Use a double-entry or append-only wallet ledger; do not treat a mutable balance field as the only financial record.
- On payment timeout, query provider status before marking an order failed. Late successful payments must be reconciled.
- Define an explicit refund path for duplicate payments, wrong player IDs, failed delivery, and cancelled orders. Do not promise refunds where a provider or game policy prohibits them.
- Protect player IDs and payment references; mask sensitive details in admin screens and logs.
- Admin manual adjustments require a reason, operator identity, timestamp, and immutable audit entry.

## 5. Tournament Lifecycle

- Draft → Published/Upcoming → Registration Open → Registration Closed → Check-in → Live → Results Pending Review → Completed. Cancellation is a separate terminal path with refund/reward handling.
- Each tournament defines game, title, banner, description, region/server, mode (solo/duo/squad), max slots, entry fee, prize rules, registration deadline, start time/time zone, check-in window, match rules, scoring rules, and room-information release policy.

- Joining requires an eligible account, open registration, available capacity, and sufficient wallet balance for paid entries. Deduct entry fees atomically and issue a registration receipt.
- Prevent duplicate registration for the same user/team and tournament. For teams, validate roster size and prevent a player from joining conflicting teams.
- Use a waitlist when enabled. If a registration is cancelled, apply the published cancellation policy and release the slot safely.
- Room ID/password should be visible only to eligible registered participants and only at the configured release time. Avoid exposing room credentials in public APIs, page source, or notifications sent to nonparticipants.
- Admin result entry must include evidence/reference notes and a reviewer. High-value prizes should support dual approval.

## 6. Tournament Scoring and Rewards

- Support configurable scoring per game: placement points, kills/assists, round wins, tie-break rules, penalties, and disqualification.
- Store raw match results separately from calculated standings. Recalculate standings deterministically whenever approved result data changes.
- Display provisional results until an authorized admin approves them. Provide a dispute window and a process to review score evidence.
- Prize distribution should be a separate tracked workflow: Pending Approval → Approved → Processing → Paid, or Failed/Under Review. Record recipient, amount, transaction reference, and operator.
- Do not advertise a guaranteed prize pool unless funding and terms are confirmed. Publish the prize split and eligibility rules before registration opens.

## 7. Admin Dashboard

- Overview: total orders, pending payments, delivery failures, active tournaments, registration counts, disputed results, and pending payouts.
- Top-Up Management: games, packages, price/cost, stock or provider availability, delivery configuration, orders, payment reconciliation, refunds, retries, and manual review.
- Tournament Management: create/edit/publish, registration limits, schedule, rules, teams, check-in, room credentials, match rounds, scoring configuration, cancellation, and participant communications.
- Results and Rewards: enter/import results, review evidence, approve standings, handle disputes, approve prizes, and reconcile payout status.
- Users and Risk: account status, abuse reports, duplicate-account signals, suspicious payment patterns, and tournament bans, with documented reasons and appeal path.
- Permissions: role-based access such as Support, Top-Up Operator, Tournament Manager, Finance Reviewer, and Super Admin. Restrict sensitive financial actions and room credentials by role.
- Audit log: record who did what, when, the affected object, before/after values where appropriate, reason, and request/correlation ID.

## 8. Suggested Database Entities

- users(id, display_name, email/phone, status, created_at)
- wallets(user_id, currency, available_balance, locked_balance) — balances are derived/maintained transactionally and reconciled against the ledger.
- wallet_ledger(id, user_id, type, amount, direction, reference_type, reference_id, idempotency_key, created_at)
- topup_games(id, name, active, delivery_method, required_player_fields)
- topup_packages(id, game_id, name, item_quantity, price, active, sort_order)
- topup_orders(id, user_id, game_id, package_id, player_details, amount, status, idempotency_key, created_at, updated_at)
- payments(id, order_id, provider, provider_transaction_id, amount, status, verified_at, raw_event_reference) with a unique constraint on provider + provider_transaction_id.
- tournaments(id, title, game_id, mode, entry_fee, prize_pool, max_slots, starts_at, registration_closes_at, status, rules, scoring_config, room_release_at)

- tournament_registrations(id, tournament_id, user_id/team_id, status, entry_payment_ledger_id, registered_at) with unique constraints to prevent duplicates.
- teams(id, tournament_id, name, captain_user_id); team_members(team_id, user_id) with tournament-specific uniqueness rules.
- matches(id, tournament_id, round_number, match_code, scheduled_at, status, room_secret_reference)
- match_results(id, match_id, user_id/team_id, placement, kills, points, evidence_reference, submitted_by, approved_by, status)
- prize_payouts(id, tournament_id, user_id/team_id, amount, status, provider_reference, approved_by, paid_at)
- audit_logs(id, actor_id, action, entity_type, entity_id, reason, metadata, created_at)

## 9. API Outline

- Authentication: POST /api/auth/login; POST /api/auth/refresh; POST /api/auth/logout.
- Top-Up: GET /api/topups/games; GET /api/topups/games/{gameId}/packages; POST /api/topups/orders; GET /api/topups/orders/{orderId}; POST /api/payments/{provider}/webhook.
- Wallet: GET /api/wallet; GET /api/wallet/transactions; POST /api/wallet/topups.
- Tournaments: GET /api/tournaments; GET /api/tournaments/{id}; POST /api/tournaments/{id}/register; POST /api/tournaments/{id}/cancel-registration; GET /api/me/tournaments; GET /api/tournaments/{id}/standings.
- Matches: GET /api/tournaments/{id}/matches; GET /api/matches/{id}; room secrets must be returned only after server-side eligibility and release-time checks.
- Admin examples: POST /api/admin/topup-packages; PATCH /api/admin/topup-orders/{id}; POST /api/admin/tournaments; PATCH /api/admin/tournaments/{id}; POST /api/admin/matches/{id}/results; POST /api/admin/prizes/{id}/approve.
- All state-changing routes need authentication, authorization, input validation, rate limits where appropriate, idempotency for financial actions, and structured audit logs. API paths are illustrative and can be adapted to the existing codebase.

## 10. Security, Payments and Compliance

- Use HTTPS everywhere; store provider secrets in environment/secret management, never in client apps or repositories.
- Validate payment webhook signatures and amount/currency/order matching. Use provider APIs to reconcile uncertain events.
- Use database transactions and row-level locking or equivalent concurrency controls for wallet spending, registration slots, and prize payouts.
- Enforce least-privilege roles, strong admin authentication, session expiry, and audit trails.
- Rate-limit login, order creation, registration, and support endpoints. Add abuse detection and safe error messages.
- Do not store card details unless the payment provider and compliance scope explicitly support it; prefer hosted checkout/tokenized provider flows.
- Before enabling paid-entry tournaments, confirm applicable local laws, age restrictions, payment-provider terms, game publisher rules, tax/prize obligations, and whether the model could be treated as gambling or regulated gaming in target jurisdictions. This document is a technical plan, not legal advice.

## 11. UI and UX Acceptance Criteria

- Responsive, accessible screens with loading, empty, success, failure, and retry states.
- Every order displays a unique reference, amount, current status, and next action.
- Every tournament card clearly displays entry fee (including Free), prize pool, date/time zone, mode, slots, and status.
- Disable Join when registration is closed, the user is ineligible, slots are full, or payment cannot be completed; explain why.
- Prevent double taps from creating duplicate orders or registrations.
- Room details are hidden from unregistered users and unavailable before the release time.
- Wallet history, order history, registration receipts, match results, and payout history are available to the user.
- Admin actions confirm consequential operations and show the result only after the server confirms it.

## 12. Test Plan

- Payment tests: valid payment, invalid signature, wrong amount, duplicate webhook, delayed webhook, timeout, provider outage, refund, and payment success after client timeout.
- Wallet tests: simultaneous spending, duplicate idempotency key, insufficient balance, ledger reconciliation, and failed transaction rollback.
- Tournament tests: full slots, simultaneous final-slot registrations, duplicate join, cancellation, waitlist, team roster conflicts, late check-in, and tournament cancellation.
- Security tests: unauthorized admin actions, leaked room credentials, manipulated prices, replayed webhook, IDOR on orders/registrations, and rate-limit bypass.
- Results tests: tie-breaks, edited scores, disqualification, disputes, payout approval, failed payout retry, and audit record verification.
- Release gates: all critical tests pass; no duplicate credits or charges in idempotency tests; payment and prize ledgers reconcile; admin permissions reviewed; backups and incident procedures documented.

## 13. Implementation Phases

- Phase 1 — Confirm requirements: target games, top-up type, payment provider(s), country/currency, platform, existing stack, and delivery method.
- Phase 2 — Build foundations: schema/migrations, authentication, role permissions, audit logs, wallet ledger, and shared UI components.
- Phase 3 — Top-Up MVP: package catalog, order lifecycle, payment integration, verification webhook, delivery workflow, order history, and admin reconciliation.
- Phase 4 — Tournament MVP: tournament creation, registration, entry-fee handling, slots, check-in, room release, results, standings, and admin review.
- Phase 5 — Rewards and hardening: payout workflow, disputes, alerts, load tests, security review, backups, and production monitoring.
- Phase 6 — Pilot: run free tournaments first, test payment reconciliation in a controlled environment, then enable paid features only after compliance and operational checks.

## 14. Decisions Needed Before Coding

- Project/app name and current repository or framework.
- Which games are supported and whether top-up means wallet credit, direct diamonds/coins, or both.
- Payment methods to integrate (for example, eSewa, Khalti, bank transfer, or another provider) and whether merchant credentials are already approved.
- Currency and operating country; confirm whether paid tournaments and cash prizes are permitted for the intended format.
- Solo/duo/squad formats, scoring rules, cancellation/refund policy, and who verifies results.
- Whether game top-ups are delivered through an authorized provider API or manually by an operator.

## Suggested MVP release checklist

- Top-up orders and payment states are visible to users and admins.
- Payment webhooks are verified and idempotent.
- Wallet ledger reconciles with all credits, debits, refunds, and prize awards.
- Tournament registration and final-slot allocation are concurrency-safe.
- Room credentials are restricted and time-gated.
- Result approval, disputes, payout tracking, and audit logs work.
- Legal, payment-provider, publisher, and prize rules are reviewed before paid tournaments launch.